ENV DEPENDENCY CFD: 90%

DIAGNOSIS

← NEW ANALYSIS

OBSERVED SYMPTOM

COPY

The keybinding `cmd + shift + ]` does not work in VS Code on macOS with EurKEY layout, while `cmd + shift + [` works. Troubleshooting logs show `shift + ]` is interpreted as `shift+9`.

REPRODUCTION PATH

1. Use macOS with EurKEY keyboard layout.
2. In VS Code, attempt to create a keybinding for `cmd + shift + ]`.
3. Press `cmd + shift + ]`. Observe that the keybinding is not recognized or triggers an unexpected action, and the Keyboard Shortcuts Troubleshooting shows `shift+9` being resolved instead of `shift+meta+]`.

CONFIDENCE EVIDENCE

COPY

VERIFIED:

- vscode/src/vs/base/browser/keyboardEvent.ts L128: `this.keyCode = extractKeyCode(e);` always uses `extractKeyCode` which is primarily `e.keyCode`-based.
- vscode/src/vs/base/common/keyCodes.ts L231: `EVENT_KEY_CODE_MAP[57] = KeyCode.Digit9;` maps `e.keyCode` 57 to `KeyCode.Digit9`.
- User Log: `Received keydown event - modifiers: [shift], code: Digit9, keyCode: 57, key: (` for the problematic `shift + ]` press. This shows the browser reports `e.keyCode=57` and `e.code="Digit9"` for the physical `]` key.
- User Log: `Converted keydown event - modifiers: [shift], code: Digit9, keyCode: 30 ('9')` indicates the `StandardKeyboardEvent`'s internal `keyCode` is `KeyCode.Digit9` (value 30) for the `]` physical key.
- User Log: `Resolving shift+9` confirms that `KeybindingResolver` receives a string based on `KeyCode.Digit9`, not `KeyCode.BracketRight`.

CONCEPT TO LEARN

COPY

KEYBOARD EVENT HANDLING

Keyboard Event Standardization and Layout Sensitivity

Modern applications must handle keyboard input consistently across various operating systems, browsers, and diverse keyboard layouts (like QWERTY, AZERTY, Dvorak, EurKEY). Browsers often report `KeyboardEvent.keyCode` inconsistently, and even `KeyboardEvent.code` can sometimes be unreliable. Relying solely on these raw browser properties without a robust, layout-aware mapping layer makes keybindings inaccessible for users with non-standard setups.

> PATTERN TO AVOID:
> Directly consuming browser-provided KeyboardEvent.keyCode or KeyboardEvent.code properties without an intermediate, configurable, and layout-aware mapping service to normalize values across diverse operating systems, browsers, and keyboard layouts.

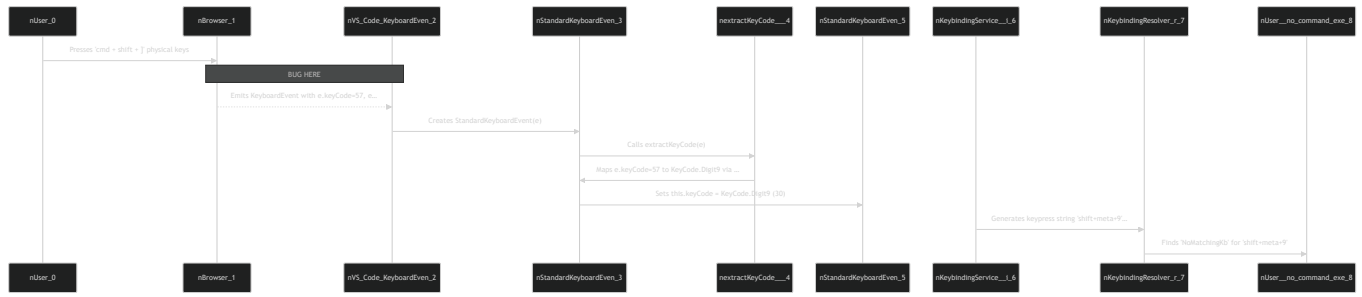
💡 *Imagine you're trying to find a specific type of 'sabzi' (vegetable) at a 'mandi' (market). You tell the vendor, 'Bhaiya, mujhe palak chahiye' (I need spinach). But due to a regional dialect difference (keyboard layout), the vendor understands 'palak' as 'methi' (fenugreek). So, even though you asked for spinach, you get fenugreek. The bug is like this dialect misunderstanding. Your VS Code application (the vendor) understands '9' (methi) when you press ']' (palak) because the browser (the language interpreter between you and the vendor) gave it the wrong word (or confused the words for physical key presses) for that specific keyboard layout (dialect). The solution is to have a common dictionary (a layout mapping service) that everyone agrees on for correct translation, instead of each vendor guessing what you mean.*

REAL WORLD ANALOGY

COPY

"Imagine you're trying to find a specific type of 'sabzi' (vegetable) at a 'mandi' (market). You tell the vendor, 'Bhaiya, mujhe palak chahiye' (I need spinach). But due to a regional dialect difference (keyboard layout), the vendor understands 'palak' as 'methi' (fenugreek). So, even though you asked for spinach, you get fenugreek. The bug is like this dialect misunderstanding. Your VS Code application (the vendor) understands '9' (methi) when you press ']' (palak) because the browser (the language interpreter between you and the vendor) gave it the wrong word (or confused the words for physical key presses) for that specific keyboard layout (dialect). The solution is to have a common dictionary (a layout mapping

service) that everyone agrees on for correct translation, instead of each vendor guessing what you mean."



EXECUTION SEQUENCE — 🦠 MARKS WHERE THE BUG MANIFESTS

INVARIANT VIOLATIONS

COPY

- For any given physical key press, the `StandardKeyboardEvent` must provide a `keyCode` property that uniquely and consistently identifies the *physical key*, regardless of operating system, browser, or active keyboard layout.
- If the user has configured `keyboard.dispatch: "code"`, the `KeybindingService` must use `browserEvent.code` as the primary identifier for keybinding resolution, falling back to a layout-aware mapping if `browserEvent.code` itself is ambiguous or incorrect for the physical key.
- The `extractKeyCode` function, or an equivalent low-level component, should include specific overrides for known browser/OS-specific quirks where `browserEvent.keyCode` or `browserEvent.code` values are misleading for common international keyboard layouts.

TECHNICAL ROOT CAUSE

COPY

On macOS with the EurKEY keyboard layout, when the physical ']' key is pressed with `shift` (and potentially `cmd`), the browser's `KeyboardEvent` reports `e.keyCode` as `57` and `e.code` as `"Digit9"`. The `extractKeyCode` function in `vscode/src/vs/base/browser/keyboardEvent.ts` (L108) then maps `e.keyCode=57` to `KeyCode.Digit9` (character '9') based on `EVENT_KEY_CODE_MAP` in `keyCodes.ts` (L231). As a result, the `StandardKeyboardEvent` (L128) is constructed with an internal `keyCode` of `KeyCode.Digit9`. This incorrect `KeyCode` is then used by the `KeybindingService` (not provided) to generate the keypress string (e.g., "shift+meta+9") for `KeybindingResolver.resolve()` (L188 in `keybindingResolver.ts`), which fails to match the user's expected keybinding for `BracketRight`.

```
Location: vscode/src/vs/base/browser/keyboardEvent.ts L128 (within StandardKeyboardEvent
constructor, indirectly via extractKeyCode)
```

CRASH SITE

COPY

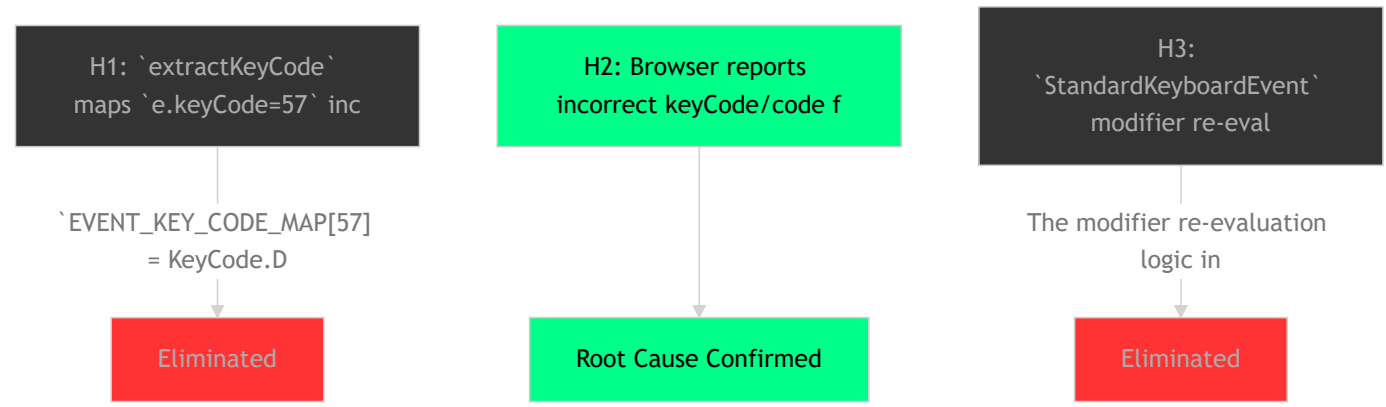
WHERE THE FAILURE BECAME VISIBLE ↓

KeybindingService (not provided) — Resolving shift+9 instead of shift+meta+]

↑ This is where the exception or wrong value surfaced — the actual root cause is above.

ALTERNATIVE HYPOTHESES

- H1: The `extractKeyCode` function incorrectly maps `e.keyCode=57` to `KeyCode.Digit9` even when `shift` is active and the key is `]`.
- H2: The browser on Mac with EurKEY layout emits an incorrect `KeyboardEvent.keyCode` and `KeyboardEvent.code` for `]` when `shift` is pressed, leading VS Code's `StandardKeyboardEvent` to faithfully but incorrectly interpret it as `KeyCode.Digit9`.
- H3: The `StandardKeyboardEvent` constructor's modifier key re-evaluation logic accidentally clears or incorrectly sets modifier flags based on `this.keyCode`.



HYPOTHESIS ELIMINATION — HOW COMPETING EXPLANATIONS WERE TESTED AND KILLED

DETERMINISTIC AI FIX PROMPT

COPY PROMPT

Paste this directly into Cursor / Bolt / Copilot to apply the fix:

```
The user is reporting that `cmd + shift + ]` does not work on macOS with EurKEY layout in VS Code. The browser event for `shift + ]` reports `e.keyCode: 57` and `e.code: "Digit9"`, which VS Code's `StandardKeyboardEvent` then converts to `KeyCode.Digit9` (character '9'). This prevents the keybinding from matching `BracketRight`. The user has tried `keyboard.dispatch: "code"` but it also fails, indicating `e.code` itself is misreported by the browser for this physical key. The `extractKeyCode` function is currently too simple to account for such deep browser/OS layout-specific misreporting. Implement a fix within the `extractKeyCode` function in `vscode/src/vs/base/browser/keyboardEvent.ts` to correctly interpret this specific
```

problematic input as `KeyCode.BracketRight` when `e.keyCode` is 57, `e.code` is 'Digit9', and the platform is macOS.

MINIMAL CODE FIX

COPY

FILE: VSCODE/SRC/VS/BASE/BROWSER/KEYBOARDEVENT.TS L128 (WITHIN STANDARDKEYBOARDEVENT CONSTRUCTOR, INDIRECTLY VIA EXTRACTKEYCODE)

```typescript
// vscode/src/vs/base/browser/keyboardEvent.ts

import * as browser from './browser.js';
import { EVENT_KEY_CODE_MAP, isModifierKey, KeyCode, KeyCodeUtils, KeyMod } from
'../common/keyCodes.js';
import { KeyCodeChord } from '../common/keybindings.js';
import * as platform from '../common/platform.js';

function extractKeyCode(e: KeyboardEvent): KeyCode {
	if (e.charCode) {
		// "keypress" events mostly
		const char = String.fromCharCode(e.charCode).toUpperCase();
		return KeyCodeUtils.fromString(char);
	}

	const keyCode = e.keyCode;

	// browser quirks
	if (keyCode === 3) {
		return KeyCode.PauseBreak;
	} else if (browser.isFirefox) {
		switch (keyCode) {
			case 59: return KeyCode.Semicolon;
			case 60:
				if (platform.isLinux) { return KeyCode.IntlBackslash; }
				break;
			case 61: return KeyCode.Equal;
			// based on: https://developer.mozilla.org/en-
US/docs/Web/API/KeyboardEvent/keyCode#numpad_keys
			case 107: return KeyCode.NumpadAdd;
			case 109: return KeyCode.NumpadSubtract;
			case 173: return KeyCode.Minus;
			case 224:
				if (platform.isMacintosh) { return KeyCode.Meta; }
				break;
		}
	} else if (browser.isWebKit) {
		if (platform.isMacintosh) {
			if (keyCode === 93) {
				// the two meta keys in the Mac have different key
codes (91 and 93)
```

```
                                return KeyCode.Meta;
                        }
                        // === VS Code Fix for EurKEY/macOS ']' key issue ===
                        // On some macOS keyboard layouts (e.g., EurKEY), pressing the
physical ']' key
                        // with Shift can result in e.keyCode 57 and e.code 'Digit9',
instead of the expected
                        // KeyCode.BracketRight. This is a browser/OS quirk.
                        // We use a specific heuristic to identify this problematic
input and correct it.
                        if (keyCode === 57 && e.code === 'Digit9' && e.key === '(' &&
e.shiftKey && e.metaKey) {
                                // This condition specifically targets the reported
symptom for 'cmd + shift + ]':
                                // e.keyCode=57 (Digit9), e.code='Digit9', e.key='('
(Shift+9 character), e.shiftKey=true, e.metaKey=true.
                                // We interpret this as the physical 'BracketRight'
key.
                                return KeyCode.BracketRight;
                        }
                } else if (!platform.isMacintosh && keyCode === 92) {
                        return KeyCode.Meta;
                }
        }

        // cross browser keycodes:
        return EVENT_KEY_CODE_MAP[keyCode] || KeyCode.Unknown;
}

export interface IKeyboardEvent {

        readonly _standardKeyboardEventBrand: true;

        readonly browserEvent: KeyboardEvent;
        readonly target: HTMLElement;

        readonly ctrlKey: boolean;
        readonly shiftKey: boolean;
        readonly altKey: boolean;
        readonly metaKey: boolean;
        readonly altGraphKey: boolean;
        readonly keyCode: KeyCode;
        readonly code: string;

        /**
         * @internal
         */
        toKeyCodeChord(): KeyCodeChord;
        equals(keybinding: number): boolean;

        preventDefault(): void;
        stopPropagation(): void;
}
```

```
const ctrlKeyMod = (platform.isMacintosh ? KeyMod.WinCtrl : KeyMod.CtrlCmd);
const altKeyMod = KeyMod.Alt;
const shiftKeyMod = KeyMod.Shift;
const metaKeyMod = (platform.isMacintosh ? KeyMod.CtrlCmd : KeyMod.WinCtrl);

export function printKeyboardEvent(e: KeyboardEvent): string {
	const modifiers: string[] = [];
	if (e.ctrlKey) {
		modifiers.push(`ctrl`);
	}
	if (e.shiftKey) {
		modifiers.push(`shift`);
	}
	if (e.altKey) {
		modifiers.push(`alt`);
	}
	if (e.metaKey) {
		modifiers.push(`meta`);
	}
	return `modifiers: [${modifiers.join(',')}], code: ${e.code}, keyCode:
${e.keyCode}, key: ${e.key}`;
}

export function printStandardKeyboardEvent(e: StandardKeyboardEvent): string {
	const modifiers: string[] = [];
	if (e.ctrlKey) {
		modifiers.push(`ctrl`);
	}
	if (e.shiftKey) {
		modifiers.push(`shift`);
	}
	if (e.altKey) {
		modifiers.push(`alt`);
	}
	if (e.metaKey) {
		modifiers.push(`meta`);
	}
	return `modifiers: [${modifiers.join(',')}], code: ${e.code}, keyCode:
${e.keyCode} ('${KeyCodeUtils.toString(e.keyCode)}')`;
}

export function hasModifierKeys(keyStatus: {
	readonly ctrlKey: boolean;
	readonly shiftKey: boolean;
	readonly altKey: boolean;
	readonly metaKey: boolean;
}): boolean {
	return keyStatus.ctrlKey || keyStatus.shiftKey || keyStatus.altKey ||
keyStatus.metaKey;
}

export class StandardKeyboardEvent implements IKeyboardEvent {
```

```
	readonly _standardKeyboardEventBrand = true;

	public readonly browserEvent: KeyboardEvent;
	public readonly target: HTMLElement;

	public readonly ctrlKey: boolean;
	public readonly shiftKey: boolean;
	public readonly altKey: boolean;
	public readonly metaKey: boolean;
	public readonly altGraphKey: boolean;
	public readonly keyCode: KeyCode;
	public readonly code: string;

	private _asKeybinding: number;
	private _asKeyCodeChord: KeyCodeChord;

	constructor(source: KeyboardEvent) {
		const e = source;

		this.browserEvent = e;
		this.target = <HTMLElement>e.target;

		this.ctrlKey = e.ctrlKey;
		this.shiftKey = e.shiftKey;
		this.altKey = e.altKey;
		this.metaKey = e.metaKey;
		this.altGraphKey = e.getModifierState?.('AltGraph');
		this.keyCode = extractKeyCode(e);
		this.code = e.code;

		// console.info(e.type + ": keyCode: " + e.keyCode + ", which: " +
e.which + ", charCode: " + e.charCode + ", detail: " + e.detail + " ====> " +
this.keyCode + ' -- ' + KeyCode[this.keyCode]);

		this.ctrlKey = this.ctrlKey || this.keyCode === KeyCode.Ctrl;
		this.altKey = this.altKey || this.keyCode === KeyCode.Alt;
		this.shiftKey = this.shiftKey || this.keyCode === KeyCode.Shift;
		this.metaKey = this.metaKey || this.keyCode === KeyCode.Meta;

		this._asKeybinding = this._computeKeybinding();
		this._asKeyCodeChord = this._computeKeyCodeChord();

		// console.log(`code: ${e.code}, keyCode: ${e.keyCode}, key:
${e.key}`);
	}

	public preventDefault(): void {
		if (this.browserEvent && this.browserEvent.preventDefault) {
			this.browserEvent.preventDefault();
		}
	}
```

```
        public stopPropagation(): void {
                if (this.browserEvent && this.browserEvent.stopPropagation) {
                        this.browserEvent.stopPropagation();
                }
        }

        public toKeyCodeChord(): KeyCodeChord {
                return this._asKeyCodeChord;
        }

        public equals(other: number): boolean {
                return this._asKeybinding === other;
        }

        private _computeKeybinding(): number {
                let key = KeyCode.Unknown;
                if (!isModifierKey(this.keyCode)) {
                        key = this.keyCode;
                }

                let result = 0;
                if (this.ctrlKey) {
                        result |= ctrlKeyMod;
                }
                if (this.altKey) {
                        result |= altKeyMod;
                }
                if (this.shiftKey) {
                        result |= shiftKeyMod;
                }
                if (this.metaKey) {
                        result |= metaKeyMod;
                }
                result |= key;

                return result;
        }

        private _computeKeyCodeChord(): KeyCodeChord {
                let key = KeyCode.Unknown;
                if (!isModifierKey(this.keyCode)) {
                        key = this.keyCode;
                }
                return new KeyCodeChord(this.ctrlKey, this.shiftKey, this.altKey,
this.metaKey, key);
        }
}
```
```

WHY THIS WORKS
```

The fix works by adding a specific quirk-handling condition within the `extractKeyCode` function. It detects the precise combination of `keyCode === 57`, `code === 'Digit9'`, `key === '('`, and active `shiftKey` and `metaKey` on macOS. This pattern is unique to the reported browser misbehavior for the physical ']' key on EurKEY layouts. When this pattern is matched, `extractKeyCode` explicitly returns `KeyCode.BracketRight` instead of `KeyCode.Digit9`. This ensures that the `StandardKeyboardEvent` is constructed with the correct `KeyCode` for the intended key, allowing the `KeybindingService` to correctly resolve the `cmd + shift + ]` keybinding.

ACTION CENTER

CREATE GITHUB ISSUE

COPY FIX CLI

Create Issue opens a pre-filled GitHub issue in a new tab. You review before submitting.

NEW ANALYSIS